Netty案例学习

ByteBuf的基本演示

```
public class NettyByteBufExample {

    public static void main(String[] args){
        ByteBuf bytebuf = ByteBufAllocator.DEFAULT.buffer();

        System.out.println("============before=============");

        log(bytebuf);
        // 构建一个字符串
        StringBuilder stringBuilder = new StringBuilder();
        for (int i = 0; i<1000; i++){
            stringBuilder.append("-"+i);
        }
        bytebuf.writeBytes(stringBuilder.toString().getBytes());
        System.out.println("==========after===========");
        log(bytebuf);
    }

    private static void log(ByteBuf bytebuf) {

        StringBuilder stringBuilder = new StringBuilder();
        stringBuilder.append(" read index").append(bytebuf.readerIndex());// 读索引
        stringBuilder.append(" write index").append(bytebuf.writerIndex());// 写索引
        stringBuilder.append(" capacity : ").append(bytebuf.capacity());// 容量

        ByteBufUtil.appendPrettyHexDump(stringBuilder,bytebuf);

        System.out.println(stringBuilder.toString());

    }
}
```

ByteBuf的创建

```java
public class ByteBufCreateExample {

    public static void main(String[] args) {
        // 由jvm管理内存
        ByteBuf buf = ByteBufAllocator.DEFAULT.heapBuffer();// 堆内存
        System.out.println(buf);

        // 堆外内存
        ByteBufAllocator.DEFAULT.directBuffer();
        // JVM: -Dio.netty.allocator.type={unpooled | polled}
        // byteBuf使用了池化技术
    }
}
```

ByteBuf常用方法

```java
public class NettyByteBufExample {

    public static void main(String[] args) {
        ByteBuf bytebuf = ByteBufAllocator.DEFAULT.heapBuffer(); // 堆内存

        bytebuf.writeBytes(new byte[]{1, 2, 3, 4});
        log(bytebuf);
        bytebuf.writeInt(6);
        log(bytebuf);
        System.out.println("开始读操作");
        bytebuf.markReaderIndex();// 标记索引位置，为了将来可以回到这里
        byte b = bytebuf.readByte();
        System.out.println(b);
        bytebuf.resetReaderIndex();// 重新回到标记的位置
        log(bytebuf);
    }

    private static void log(ByteBuf bytebuf) {

        StringBuilder stringBuilder = new StringBuilder();
        stringBuilder.append(" read index").append(bytebuf.readerIndex());// 读索引
        stringBuilder.append(" write index").append(bytebuf.writerIndex());// 写索引
        stringBuilder.append(" capacity : ").append(bytebuf.capacity());// 容量
        // 格式化成16进制
        ByteBufUtil.appendPrettyHexDump(stringBuilder, bytebuf);

        System.out.println(stringBuilder.toString());

    }
}
```

Bytebuf零拷贝

```java
// 拆分
public class ByteBufCreateExample {
```

```java
    public static void main(String[] args) {
        // 由jvm管理内存
        ByteBuf buf = ByteBufAllocator.DEFAULT.buffer();

        buf.writeBytes(new byte[]{1,2,3,4,5,6,7,8,9,10});

        ByteBuf buf1 = buf.slice(0, 5);
        ByteBuf buf2 = buf.slice(5, 5);
        log(buf1);
        log(buf2);
        buf.setByte(3,9);
        log(buf1);
    }

    private static void log(ByteBuf bytebuf) {

        StringBuilder stringBuilder = new StringBuilder();
        stringBuilder.append(" read index").append(bytebuf.readerIndex());// 读索引
        stringBuilder.append(" write index").append(bytebuf.writerIndex());// 写索引
        stringBuilder.append(" capacity : ").append(bytebuf.capacity());// 容量

        ByteBufUtil.appendPrettyHexDump(stringBuilder, bytebuf);

        System.out.println(stringBuilder.toString());

    }
}

// 合并
public class CompositeByteBufExample{
    public static void main(String[] args){
        ByteBuf header = ByteBufAllocator.DEFAULT.buffer();// 定义一个buffer
        header.writeBytes(new byte[]{1,2,3,4,5});
        ByteBuf body = ByteBufAllocator.DEFAULT.buffer();// 定义一个buffer
        body.writeBytes(new byte[]{1,2,3,4,5});
        // 把两个合并起来 第一种方式
        Bytebuf total = Unpooled.buffer(header.readableBytes()+body.readableBytes());
        total.writeBytes(header);
        total.writeBytes(body);
        // 复合buffer,零拷贝机制，只是把两个buffer合并，并不会去重新创建一个buffer
        ComositeByteBuf compositeByteBuf = Unpooled.compositeBuffer();
        compositeByteBuf.addComponents(true,header,body);
        log(compositeByteBuf);
        // 第三种方式
        ByteBuf total = Unpooled.wrappedBuffer(header,body);
        log(total);
    }

    private static void log(ByteBuf bytebuf) {

        StringBuilder stringBuilder = new StringBuilder();

        stringBuilder.append(" read index").append(bytebuf.readerIndex());// 读索引
```

```
        stringBuilder.append(" write index").append(bytebuf.writerIndex());// 写索引
        stringBuilder.append(" capacity : ").append(bytebuf.capacity());// 容量

        ByteBufUtil.appendPrettyHexDump(stringBuilder, bytebuf);

        System.out.println(stringBuilder.toString());

    }
}
```

netty心跳案例

假如一个用户他长时间没有读或者没有写数据的话，那么这个连接就一直占用着资源，所以我们一般都会强制这个用户下线

心跳实现代码

服务端代码

```
public class Server {

    public static void main(String[] args) {
        //服务类
        ServerBootstrap bootstrap = new ServerBootstrap();

        //boss和worker
        EventLoopGroup boss = new NioEventLoopGroup();
        EventLoopGroup worker = new NioEventLoopGroup();

        try {
            //设置线程池
            bootstrap.group(boss, worker)
            //设置socket工厂、
            .channel(NioServerSocketChannel.class)
            //设置管道工厂
            .childHandler(new ChannelInitializer<Channel>() {

                @Override
                protected void initChannel(Channel ch) throws Exception {
                    ch.pipeline().addLast(new StringDecoder());
                    ch.pipeline().addLast(new StringEncoder());
                }
            })
            //设置参数，TCP参数
            .option(ChannelOption.SO_BACKLOG, 2048)//serverSocketchannel的设置，链接缓冲池的大小
            .childOption(ChannelOption.SO_KEEPALIVE, true)//socketchannel的设置,维持链接的活跃，清除死链接
            .childOption(ChannelOption.TCP_NODELAY, true);//socketchannel的设置,关闭延迟发送

            //绑定端口

            ChannelFuture future = bootstrap.bind(8080);
```

```
            System.out.println("start");

            //等待服务端关闭
            future.channel().closeFuture().sync();

        } catch (Exception e) {
            e.printStackTrace();
        } finally{
            //释放资源
            boss.shutdownGracefully();
            worker.shutdownGracefully();
        }
    }
}
```

新建一个handler来处理业务逻辑

```
public class ServerHandler extends SimpleChannelInboundHandler<String> {

    @Override
    protected void channelRead0(ChannelHandlerContext ctx, String msg) throws Exception {
        System.out.println(msg);

//      ctx.channel().writeAndFlush("hi");
        ctx.writeAndFlush("hi");
    }

    @Override
    public void userEventTriggered(final ChannelHandlerContext ctx, Object evt) throws
Exception {
        if(evt instanceof IdleStateEvent){
            IdleStateEvent event = (IdleStateEvent)evt;
            // 如果你既没有读，也没有写
            if(event.state() == IdleState.ALL_IDLE){

                //清除超时会话，给客户端返回提示信息
                ChannelFuture writeAndFlush = ctx.writeAndFlush("you will close");
                writeAndFlush.addListener(new ChannelFutureListener() {

                    @Override
                    public void operationComplete(ChannelFuture future) throws Exception {
                        // 在关闭会话，提玩家下线
                        ctx.channel().close();
                    }
                });
            }
        }else{
            super.userEventTriggered(ctx, evt);
        }
    }
```

```
    /**
     * 新客户端接入
     */
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelActive");
    }

    /**
     * 客户端断开
     */
    @Override
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("channelInactive");
    }

    /**
     * 异常
     */
    @Override
    public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause) throws
Exception {
        cause.printStackTrace();
    }


}
```

给pipeline绑定业务handler,同时要去检测心跳，netty提供额一个类IdleStateHandler

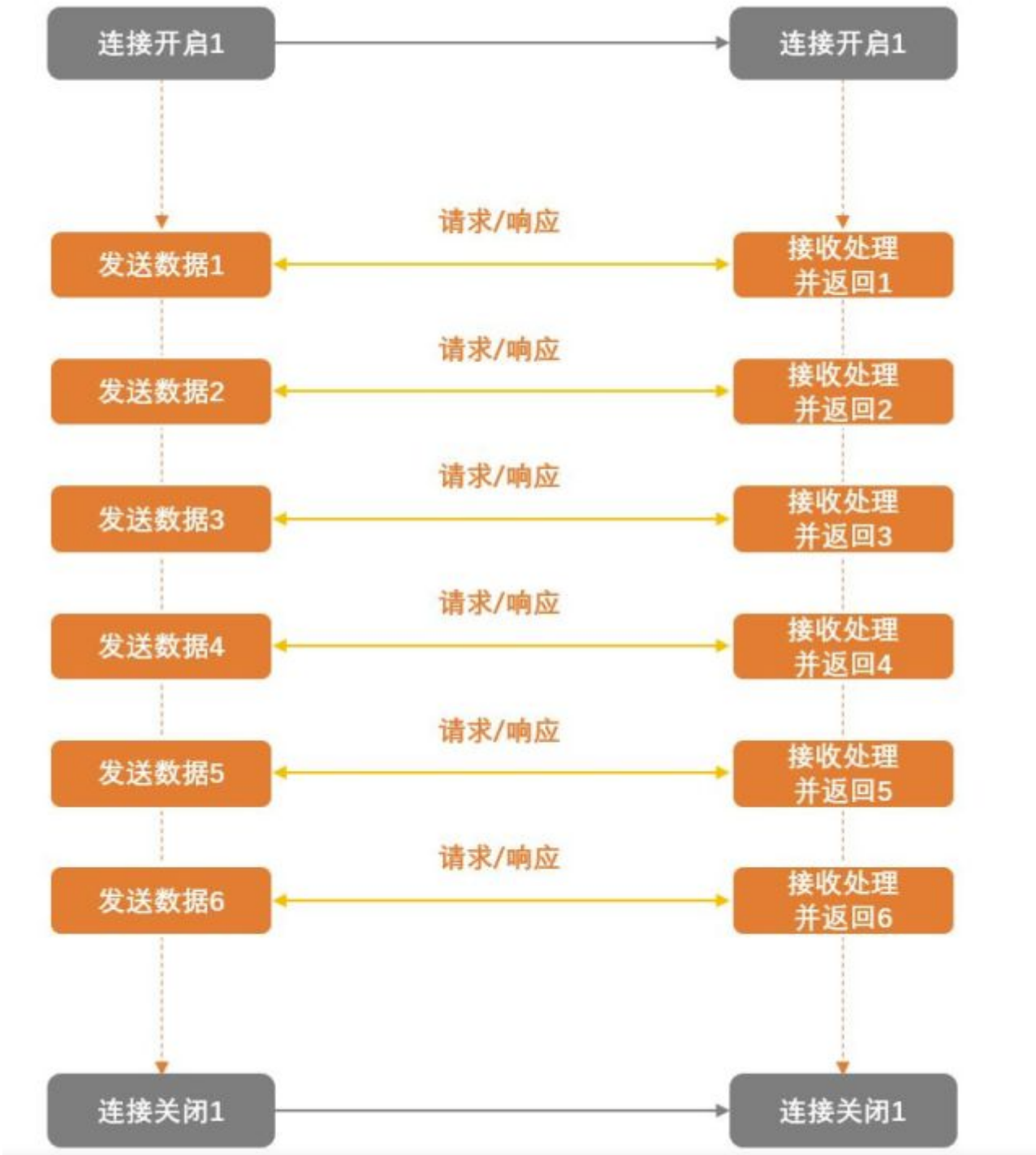
```
@Override
protected void initChannel(Channel ch) throws Exception {
    // 检测心跳handler，第一个参数是5秒内没有读操作，第二个参数5秒内没有写操作，第三个参数，10内没有读写
操作
    ch.pipeline().addLast(new IdleStateHandler(5, 5, 10));
    ch.pipeline().addLast(new StringDecoder());
    ch.pipeline().addLast(new StringEncoder());
    ch.pipeline().addLast(new ServerHandler());
}
```

启动服务端,打开客户端，模拟客户端请求

telnet 192.168.8.28 8080

netty长链接案例

- 什么是长链接：长连接意味着进行一次数据传输后，不关闭连接，长期保持连通状态。如果两个应用程序之间有新的数据需要传输，则直接复用这个连接，无需再建立一个新的连接



- 什么是短连接：短连接意味着每一次的数据传输都需要建立一个新的连接，用完再马上关闭它。下次再用的时候重新建立一个新的连接，如此反复

它的优势是由于每次使用的连接都是新建的，所以基本上只要能够建立连接，数据就大概率能送达到对方。并且哪怕这次传输出现异常也不用担心影响后续新的数据传输，因为届时又是一个新的连接。缺点是每个连接都需要经过三次握手和四次握手的过程，耗时大大增加。

另外，短连接还有一个致命的缺点。我们回到前面提到的维基百科对 socket 的定义，其中说到 socket 包含通信协议、目标地址、状态等。实际当你在基于 socket 进行开发的时候，这些包含的具体资源主要就是这 5 个：源 IP、源端口、目的 IP、目的端口、协议，有个专业的叫法称之为“五元组”。在一台计算机上只要这五元组的值不重复，那么连接就可以被建立。然而一台计算机最多只能开启 65535 个端口，如果现在两个进程之间需要通信，作为服务端的 IP 和端口必然是固定的，因此单个客户端理论上最多只能与服务端同时建立 65535 个 socket 连接。如果除去操作系统和其它进程所占用的端口，实际还会更少。所以，一旦使用不当，在很短的时间内建立了大量连接，端口很容易被占用完。这不但会导致自身无法正常工作，还会影响到同一台计算机上的其它进程。

- socket实现长链接和短连接
  - 短连接

```
public class ClientSocketShortLink {
    public static void main(String[] args) throws IOException {
        Socket socket = new Socket("localhost",8080);

        BufferedWriter bufferedWriter = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
        bufferedWriter.write("我是客户端\n");
        bufferedWriter.flush();

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        String s = bufferedReader.readLine();
        System.out.println(s);
        bufferedReader.close();

        bufferedWriter.close();
```

```
    }
}

public class ServerSocketShortLink {

    public static void main(String[] args) throws Exception {
        ServerSocket serverSocket = new ServerSocket(8080);

        Socket socket = serverSocket.accept();

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));//输入流
        String clientStr = bufferedReader.readLine(); //被阻塞了
        System.out.println("接收到客户端的信息: " + clientStr);

        BufferedWriter bufferedWriter = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
        bufferedWriter.write("我收到了信息\n");
        bufferedWriter.flush();

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

- 长链接

```
public class ServerSocketLonglink {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(8080);
        Socket socket = null;
        while (true) {
            socket = serverSocket.accept();

            BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));//输入流
            String clientStr = bufferedReader.readLine(); //被阻塞了
            System.out.println("接收到客户端的信息: " + clientStr);

            BufferedWriter bufferedWriter = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
            bufferedWriter.write("我收到了信息\n");
            bufferedWriter.flush();
        }

    }
}

public class ClientSocketLongLink {
    public static void main(String[] args) throws IOException {
        while (true){
            Socket socket = new Socket("localhost",8080);
```

```
            BufferedWriter bufferedWriter = new BufferedWriter(new
OutputStreamWriter(socket.getOutputStream()));
            bufferedWriter.write("我是客户端\n");
            bufferedWriter.flush();

            BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            String s = bufferedReader.readLine();
            System.out.println(s);
        }

    }
}
```

- 基于netty实现长链接

```
<!-- 解码and编码器 -->
<!-- https://mvnrepository.com/artifact/org.msgpack/msgpack -->
<dependency>
    <groupId>org.msgpack</groupId>
    <artifactId>msgpack</artifactId>
    <version>0.6.12</version>
</dependency>
<!-- netty 核心依赖 -->
<!-- https://mvnrepository.com/artifact/io.netty/netty-all -->
<dependency>
    <groupId>io.netty</groupId>
    <artifactId>netty-all</artifactId>
    <version>4.1.6.Final</version>
</dependency>
```

  - 创建配置模型`model(模型类) ,  TypeData(参数配置类)`

```
@Message
public class Model implements Serializable{

    private static final long serialVersionUID = 1L;

    //类型
    private int type;

    //内容
    private String body;

    public String getBody() {
        return body;
    }

    public void setBody(String body) {
        this.body = body;

    }
```

```
    public int getType() {
        return type;
    }

    public void setType(int type) {
        this.type = type;
    }

    @Override
    public String toString() {
        return "Model [type=" + type + ", body=" + body + "]";
    }
}

public interface TypeData {
    // 连接
    byte PING = 1;
    // 返回
    byte PONG = 2;
    //业务处理
    byte CUSTOMER = 3;
}
```

- MsgPckDecode(解码器)

```
public class MsgPckDecode extends MessageToMessageDecoder<ByteBuf>{

    @Override
    protected void decode(ChannelHandlerContext ctx, ByteBuf msg,
            List<Object> out) throws Exception {

        final  byte[] array;
        // 获取到消息的长度
        final int length = msg.readableBytes();
        // 定义存放消息byte
        array = new byte[length];
        // 把消息写到array中
        msg.getBytes(msg.readerIndex(), array, 0, length);

        MessagePack pack = new MessagePack();
        // 把消息转成Model.class添加到list中
        out.add(pack.read(array, Model.class));

    }
}
```

- MsgPckEncode(编码器)

```
public class MsgPckEncode extends MessageToByteEncoder<Object>{

    @Override
    protected void encode(ChannelHandlerContext ctx, Object msg, ByteBuf buf)
            throws Exception {
        MessagePack pack = new MessagePack();
        // 得到消息的byte数组
        byte[] write = pack.write(msg);
        // 然后写入到bufer中
        buf.writeBytes(write);

    }
}
```

- 创建公用的控制器 `Middleware` 这就相当于一个枢纽站

```
public abstract class Middleware extends ChannelInboundHandlerAdapter{

    protected String name;
    //记录次数
    private int heartbeatCount = 0;

    //获取server and client 传入的值
    public Middleware(String name) {
        this.name = name;
    }
    /**
     *继承ChannelInboundHandlerAdapter实现了channelRead就会监听到通道里面的消息
     */
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg)
            throws Exception {
        Model m = (Model) msg;
        int type = m.getType();
        switch (type) {
        case 1:
            sendPongMsg(ctx);
            break;
        case 2:
            System.out.println(name + " get  pong  msg  from" +
ctx.channel().remoteAddress());
            break;
        case 3:
            handlerData(ctx,msg);
            break;
        default:
            break;
        }
    }

    protected abstract void handlerData(ChannelHandlerContext ctx,Object msg);
```

```
    protected void sendPingMsg(ChannelHandlerContext ctx){
        Model model = new Model();

        model.setType(TypeData.PING);

        ctx.channel().writeAndFlush(model);

        heartbeatCount++;

        System.out.println(name + " send ping msg to " +
ctx.channel().remoteAddress() + "count :" + heartbeatCount);
    }

    private void sendPongMsg(ChannelHandlerContext ctx) {

        Model model = new Model();

        model.setType(TypeData.PONG);

        ctx.channel().writeAndFlush(model);

        heartbeatCount++;

        System.out.println(name +" send pong msg to "+ctx.channel().remoteAddress()
+" , count :" + heartbeatCount);
    }

    @Override
    public void userEventTriggered(ChannelHandlerContext ctx, Object evt)
            throws Exception {
        IdleStateEvent stateEvent = (IdleStateEvent) evt;

        switch (stateEvent.state()) {
        // 如果是读
        case READER_IDLE:
            handlerReaderIdle(ctx);
            break;
        // 如果是写
        case WRITER_IDLE:
            handlerWriterIdle(ctx);
            break;
        // 读或者写
        case ALL_IDLE:
            handlerAllIdle(ctx);
            break;
        default:
            break;
        }
    }

    protected void handlerAllIdle(ChannelHandlerContext ctx) {

        System.err.println("---ALL_IDLE---");
```

```
    }

    protected void handlerWriterIdle(ChannelHandlerContext ctx) {
        System.err.println("---WRITER_IDLE---");
    }


    protected void handlerReaderIdle(ChannelHandlerContext ctx) {
        System.err.println("---READER_IDLE---");
    }

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.err.println(" ---"+ctx.channel().remoteAddress() +"----- is  action"
);
    }

    @Override
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {
        System.err.println(" ---"+ctx.channel().remoteAddress() +"----- is
inAction");
    }
}
```

- 创建client 客户端

```
public class Client {

    private NioEventLoopGroup worker = new NioEventLoopGroup();

    private Channel channel;

    private Bootstrap bootstrap;

    public static void main(String[] args) {
        Client  client = new Client();

        client.start();

        client.sendData();
    }

    private void start() {
        bootstrap = new Bootstrap();
        bootstrap.group(worker)
        .channel(NioSocketChannel.class)
        .option(ChannelOption.TCP_NODELAY, true)
        .handler(new ChannelInitializer<Channel>() {
            @Override
            protected void initChannel(Channel ch) throws Exception {
                ChannelPipeline pipeline = ch.pipeline();

                pipeline.addLast(new IdleStateHandler(0,0,5));
```

```
                pipeline.addLast(new MsgPckDecode());

                pipeline.addLast(new MsgPckEncode());

                pipeline.addLast(new Client3Handler(Client.this));
            }
        });
        doConnect();
    }

    /**
     * 连接服务端 and 重连
     */
    protected void doConnect() {

        if (channel != null && channel.isActive()){
            return;
        }
        ChannelFuture connect = bootstrap.connect("127.0.0.1", 8081);
        //实现监听通道连接的方法
        connect.addListener(new ChannelFutureListener() {

            @Override
            public void operationComplete(ChannelFuture channelFuture) throws
Exception {

                if(channelFuture.isSuccess()){
                    channel = channelFuture.channel();
                    System.out.println("连接成功");
                }else{
                    System.out.println("每隔2s重连....");
                    // 定时器
                    channelFuture.channel().eventLoop().schedule(new Runnable() {

                        @Override
                        public void run() {
                            doConnect();
                        }
                    },2,TimeUnit.SECONDS);
                }
            }
        }).sync();
    }
    /**
     * 向服务端发送消息
     */
    private void sendData() {
        Scanner sc= new Scanner(System.in);
        for (int i = 0; i < 1000; i++) {

            if(channel != null && channel.isActive()){

                //获取一个键盘扫描器
```

```
                String nextLine = sc.nextLine();
                Model model = new Model();

                model.setType(TypeData.CUSTOMER);

                model.setBody(nextLine);

                channel.writeAndFlush(model);
            }
        }
    }
}
```

- 创建客户端的handler控制器

```
public class ClientHandler extends Middleware {

    private Client client;

    public ClientHandler(Client client) {
        super("client");
        this.client = client;
    }

    @Override
    protected void handlerData(ChannelHandlerContext ctx, Object msg) {
        Model model = (Model) msg;
        System.out.println("client  收到数据:  " + model.toString());
    }
    @Override
    protected void handlerAllIdle(ChannelHandlerContext ctx) {
        super.handlerAllIdle(ctx);
        sendPingMsg(ctx);
    }
    @Override
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {
        super.channelInactive(ctx);
        client.doConnect();
    }
    @Override
    public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause)
            throws Exception {
        System.out.println(name + "exception :"+ cause.toString());
    }
}
```

- 创建服务端server

```
public class server {
```

```
    public static void main(String[] args) {
        EventLoopGroup bossGroup = new NioEventLoopGroup(1);

        EventLoopGroup workerGroup = new NioEventLoopGroup(4);
        try {
            ServerBootstrap serverBootstrap = new ServerBootstrap();

            serverBootstrap.group(bossGroup, workerGroup)
            .channel(NioServerSocketChannel.class)
            .localAddress(8081)
            .childHandler(new ChannelInitializer<Channel>() {

                @Override
                protected void initChannel(Channel ch) throws Exception {
                    ChannelPipeline pipeline = ch.pipeline();
                    // 心跳检测  检测心跳handler，第一个参数是几秒内没有读操作，第二个参数几秒内没有写操作，第三个参数，几内没有读写操作
                    pipeline.addLast(new IdleStateHandler(10,0,0));
                     // msg解码器
                    pipeline.addLast(new MsgPckDecode());
                    // msg编码器
                    pipeline.addLast(new MsgPckEncode());
                    pipeline.addLast(new serverHandler());
                }
            });
            System.out.println("start server 8081 --");
            // sync()启动端口并同步等待连接管道，不要执行到finally快中
            ChannelFuture sync = serverBootstrap.bind().sync();
            // 如果有人关闭了管道，这里就开始调用，并且执行finally中的方法关闭连接
            sync.channel().closeFuture().sync();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }finally{
            //优雅的关闭资源
            bossGroup.shutdownGracefully();
            workerGroup.shutdownGracefully();
        }
    }
}
```

- 创建serverHandler控制器

```
public class ServerHandler extends Middleware {

    public ServerHandler() {
        super("server");
        // TODO Auto-generated constructor stub
    }
    @Override
    protected void handlerData(ChannelHandlerContext ctx, Object msg) {
        Model model  = (Model) msg;
        System.out.println("server 接收数据 :  " +  model.toString());
            model.setType(TypeData.CUSTOMER);
```

```
            model.setBody("---------------");
            ctx.channel().writeAndFlush(model);
            System.out.println("server 发送数据:  " + model.toString());
    }
    @Override
    protected void handlerReaderIdle(ChannelHandlerContext ctx) {
        super.handlerReaderIdle(ctx);
        System.err.println(" ---- client "+ ctx.channel().remoteAddress().toString()
+ " reader timeOut, --- close it");
        ctx.close();
    }
    @Override
    public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause)
            throws Exception {
        System.err.println( name +"  exception" + cause.toString());
    }
}
```

先运行服务端，然后再启动客户端 会根据设置的端口连接服务端，在客户端输入消息就会得到服务端的回应，如果超过5秒没有进行读写就会触发`IdleStateHandler`类超时事件 来进行心跳包的传输，服务端未检测到客户端的读写或者心跳就会主动关闭channel通道

netty粘包黏包案例

- 网络传输中，我们并不能确定某些报文是否结束，比如客户端发送i am wentai，i am fine，服务端接收到的 i am we 然后再收到ntai,i am fine,这种就会有问题
- 建立netty的客户端和服务端
  - 服务端

```
public class NettyServer {

    public static void main(String[] args) {
        EventLoopGroup bossGroup = new NioEventLoopGroup();
        EventLoopGroup workGroup = new NioEventLoopGroup();

        ServerBootstrap serverBootstrap = new ServerBootstrap();
        serverBootstrap.group(bossGroup,workGroup)
                .channel(NioServerSocketChannel.class)
                .childHandler(new ChannelInitializer<SocketChannel>() {
                    @Override
                    protected void initChannel(SocketChannel ch) throws Exception
{
                        ByteBuf delimiter =
Unpooled.copiedBuffer("&".getBytes());
                        ch.pipeline()
                                .addLast(new SimpleServerHandler());
                    }
                });
        try {
            ChannelFuture channelFuture = serverBootstrap.bind(8080).sync();

            channelFuture.channel().closeFuture().sync();
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        }finally {
            bossGroup.shutdownGracefully();
            workGroup.shutdownGracefully();
        }
    }
}

// 业务处理hander
public class SimpleServerHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws
Exception {
        ByteBuf in = (ByteBuf) msg;
        byte[] req = new byte[in.readableBytes()];
        in.readBytes(req);
        String body = new String(req, "UTF-8");
        System.out.println("服务端接收消息: " + body);
        // 写回数据
        ByteBuf resp =
Unpooled.copiedBuffer(UUID.randomUUID().toString().getBytes());
        ctx.writeAndFlush(resp);
    }
}
```

- 客户端

```
public class NettyClient {

    public static void main(String[] args) {
        EventLoopGroup eventLoopGroup = new NioEventLoopGroup();
        Bootstrap bootstrap = new Bootstrap();
        bootstrap.group(eventLoopGroup)
                .channel(NioSocketChannel.class)
                .handler(new ChannelInitializer<SocketChannel>() {
                    @Override
                    protected void initChannel(SocketChannel ch) throws Exception
{
                        ch.pipeline()
                                .addLast(new SimpleClientHander());
                    }
                });
        try {
            ChannelFuture channelFuture =
bootstrap.connect("localhost",8080).sync();
            channelFuture.channel().closeFuture().sync();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }finally {

            eventLoopGroup.shutdownGracefully();
```

```
        }
    }
}

// 客户端处理业务handler
public class SimpleClientHander extends ChannelInboundHandlerAdapter {

    private int count;

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("客户端连接成功");
        for (int i = 0; i < 10; i++) {
            ByteBuf buf = Unpooled.copiedBuffer("客户端消息: " + i,
Charset.forName("utf-8"));

            ctx.writeAndFlush(buf);
        }
        super.channelActive(ctx);
    }

    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws
Exception {
        System.out.println("收到服务端返回的消息");
        ByteBuf buf = (ByteBuf) msg;
        byte[] data = new byte[buf.readableBytes()];
        buf.readBytes(data);
        String message = new String(data, Charset.forName("utf-8"));
        System.out.println("收到服务端的消息内容"+message);
        System.out.println("客户端收到的消息数量: " + (++count));

        super.channelRead(ctx, msg);
    }
}
```

可以发现，客户端只能收到一条消息，把10条消息当做一条消息接收了，就导致了粘包的问题

- netty中提供了4种解决粘包黏包的解码器
  - FixedLengthFrameDecoder 固定长度解码器

```
.addLast(new FixedLengthFrameDecoder(36))// 这种要确定每次发送的包不能超过36
```

- LineBasedFrameDecoder在发送消息加上换行符\n

```
ch.pipeline ().addLast ("encode", new StringEncoder ());
//如果字节超过1024还没有找到换行符则抛出异常
ch.pipeline ().addLast (new LineBasedFrameDecoder (1024));
//解码,将ByteBuf解码为String
ch.pipeline ().addLast ("decode", new StringDecoder ());
```

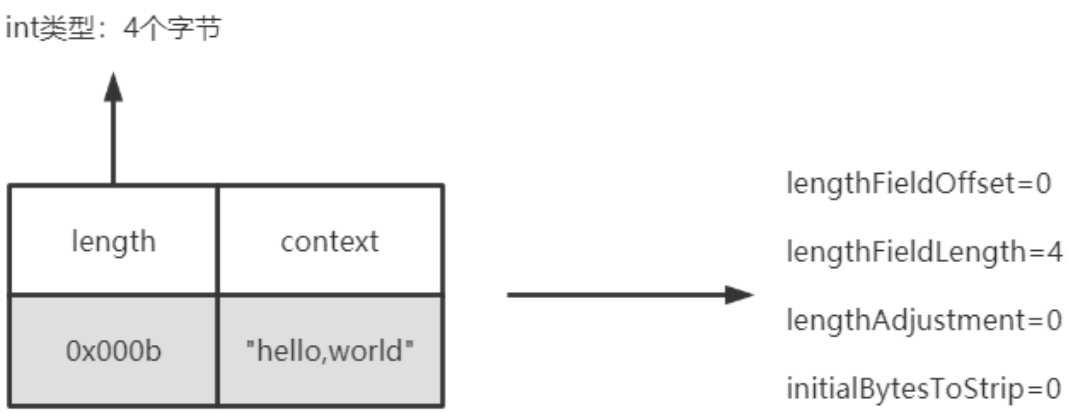
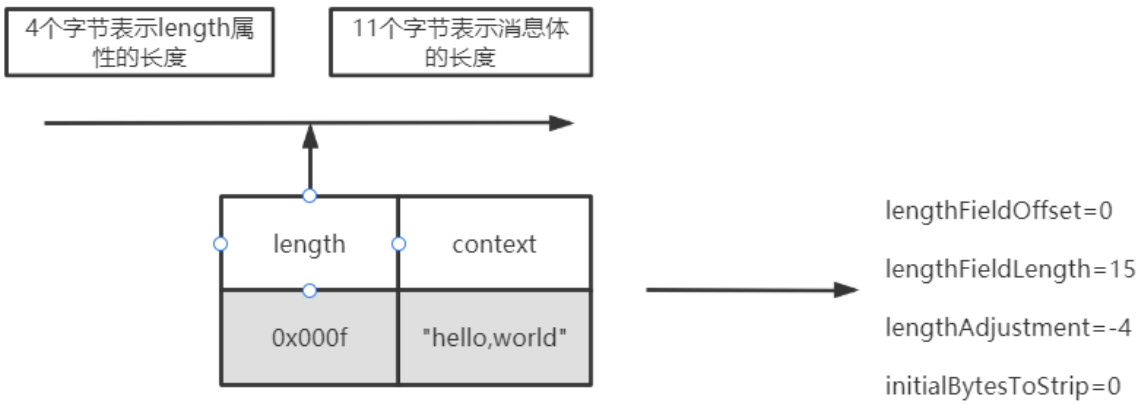
- DelimiterBasedFrameDecoder分隔符解码器

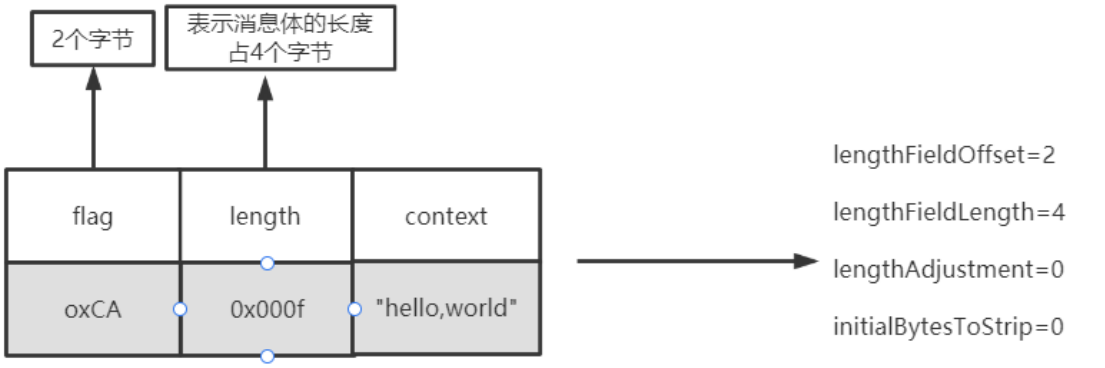
```
ByteBuf delimiter = Unpooled.copiedBuffer("&".getBytes());
// maxFrameLength:报文的最大长度
// stripDelimiter:判断解码时是否把分隔符去掉
// failFast:跟maxFrameLength联合使用，如果为true,如果大于消息的最大长度，直接抛异常，为false时等解码之后再抛出异常
// delimiter:分割符
.addLast(new DelimiterBasedFrameDecoder(10,true,true,delimiter))
```

- LengthFieldBasedFrameDecoder 长度域解码器（rocketmq就是使用的这个解码器）

  其中参数包括

```
// maxFrameLength:最大的报文长度数
// lengthFieldOffset:长度字段的偏移量，就是说多少个字段后是长度字段
// lengthFieldLength:描述消息报文的长度的字节数
// lengthAdjustment:需要调整的长度数，比如一个leng中包含了magic魔术数，还包含了长度
// initialBytesToStrip:跳过多少字节
```

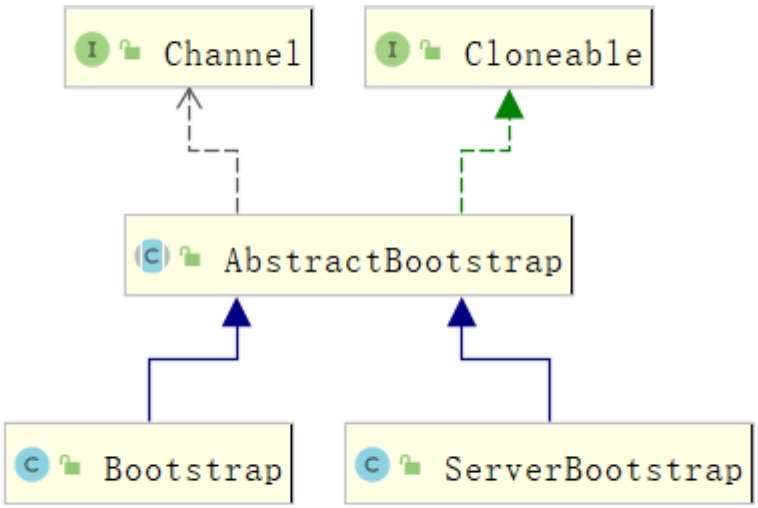
> 第一种，报文为length+context；length占4个字节，context占11个字节，几个参数的值分别为
>
> 
>
>
> 第二种，报文为length+context,但是length包含4个字节length长度和11个描述内容长度的字节
>
> 
>
>
> 第三种，报文包含flag(2个字节)+length(4个字节)+context(11个字节)



netty源码分析

AbstractBootstrap

- 首先看下ServerBootStrap的类关系图



AbstractBootstrap源码

```
// 属性配置
private static final Map.Entry<ChannelOption<?>, Object>[] EMPTY_OPTION_ARRAY = new
Map.Entry[0];
private static final Map.Entry<AttributeKey<?>, Object>[] EMPTY_ATTRIBUTE_ARRAY = new
Map.Entry[0];
// 通过工厂得到channel,为什么放在抽象类，因为对于服务端是构建服务端的channel，对于客户端构建的是客户端的
channel
private volatile ChannelFactory<? extends C> channelFactory;
// boss线程池
volatile EventLoopGroup group;
// 处理具体的handler任务
 private volatile ChannelHandler handler;
```

serverBootstrap源码

```
// 这个是封装了一层serverBootstrap，在这个类中能拿到serverBootstrap中的信息
private final ServerBootstrapConfig config = new ServerBootstrapConfig(this);
// 这个就是一个worker线程
private volatile EventLoopGroup childGroup;
// 构建worker的handler
private volatile ChannelHandler childHandler;
```

在group（）方法中

```
public ServerBootstrap group(EventLoopGroup parentGroup, EventLoopGroup childGroup) {
    // 把boss线程池进行赋值，这个地方传进去了accept事件
    super.group(parentGroup);
    if (this.childGroup != null) {
        throw new IllegalStateException("childGroup set already");
    }
    // 赋值当前woker线程池
    this.childGroup = ObjectUtil.checkNotNull(childGroup, "childGroup");
    return this;
}
```

channel方法中

```
public B channel(Class<? extends C> channelClass) {
    // 设置一个可以得到channel的工厂
    return channelFactory(new ReflectiveChannelFactory<C>(
        ObjectUtil.checkNotNull(channelClass, "channelClass")
    ));
}
```

bind方法（）

- 初始化NioServerSocketChannel
- 初始化EventLoop

- NioServerSocketChannel绑定监听端口
- NioServerSocketChannel注册到selector
- NioServerSocketChannel中的handler构建
- NioSocketChannel中的handler构建

```
public ChannelFuture bind(SocketAddress localAddress) {
    // 校验group是不是为空，工厂是不是为空channelFactory
    validate();
    return doBind(ObjectUtil.checkNotNull(localAddress, "localAddress"));
}

public B validate() {
    if (group == null) {
        throw new IllegalStateException("group not set");
    }
    if (channelFactory == null) {
        throw new IllegalStateException("channel or channelFactory not set");
    }
    return self();
}
```

dobind()

```
private ChannelFuture doBind(final SocketAddress localAddress) {
        // 初始化和注册，初始化NioServerSocketChannel注册到selector上面
        // ChannelFuture代表异步操作
        final ChannelFuture regFuture = initAndRegister();
        final Channel channel = regFuture.channel();
        if (regFuture.cause() != null) {
            return regFuture;
        }

        if (regFuture.isDone()) {
            ChannelPromise promise = channel.newPromise();
            doBind0(regFuture, channel, localAddress, promise);
            return promise;
        } else {
            final PendingRegistrationPromise promise = new
PendingRegistrationPromise(channel);
            regFuture.addListener(new ChannelFutureListener() {
                @Override
                public void operationComplete(ChannelFuture future) throws Exception {
                    Throwable cause = future.cause();
                    if (cause != null) {
                        promise.setFailure(cause);
                    } else {
                        promise.registered();

                        doBind0(regFuture, channel, localAddress, promise);
                    }
                }
```

```
            });
            return promise;
        }
    }
```

initAndRegister

```
final ChannelFuture initAndRegister() {
        Channel channel = null;
        try {
            // 这个channelFactor是ReflectiveChannelFactory，其实就是完成ServerSocketChannel的初始化，然后去给ServerSocketChannel注册连接事件，也会构建一个pipele
            channel = channelFactory.newChannel();
            // 给NioServerSocketChannel添加handler,其中一定包含一个处理accetor事件的handler
            init(channel);
        } catch (Throwable t) {
            if (channel != null) {
                channel.unsafe().closeForcibly();
                return new DefaultChannelPromise(channel,
GlobalEventExecutor.INSTANCE).setFailure(t);
            }
            return new DefaultChannelPromise(new FailedChannel(),
GlobalEventExecutor.INSTANCE).setFailure(t);
        }
        // 注册到NioEventLoop (Selector) 中 (BossNioEventLoop->线程)
        ChannelFuture regFuture = config().group().register(channel);
        // 如果有异常
        if (regFuture.cause() != null) {
            // 已经注册，则关闭
            if (channel.isRegistered()) {
                channel.close();
            } else {
                // 强制关闭
                channel.unsafe().closeForcibly();
            }
        }
        return regFuture;
    }
```

init()

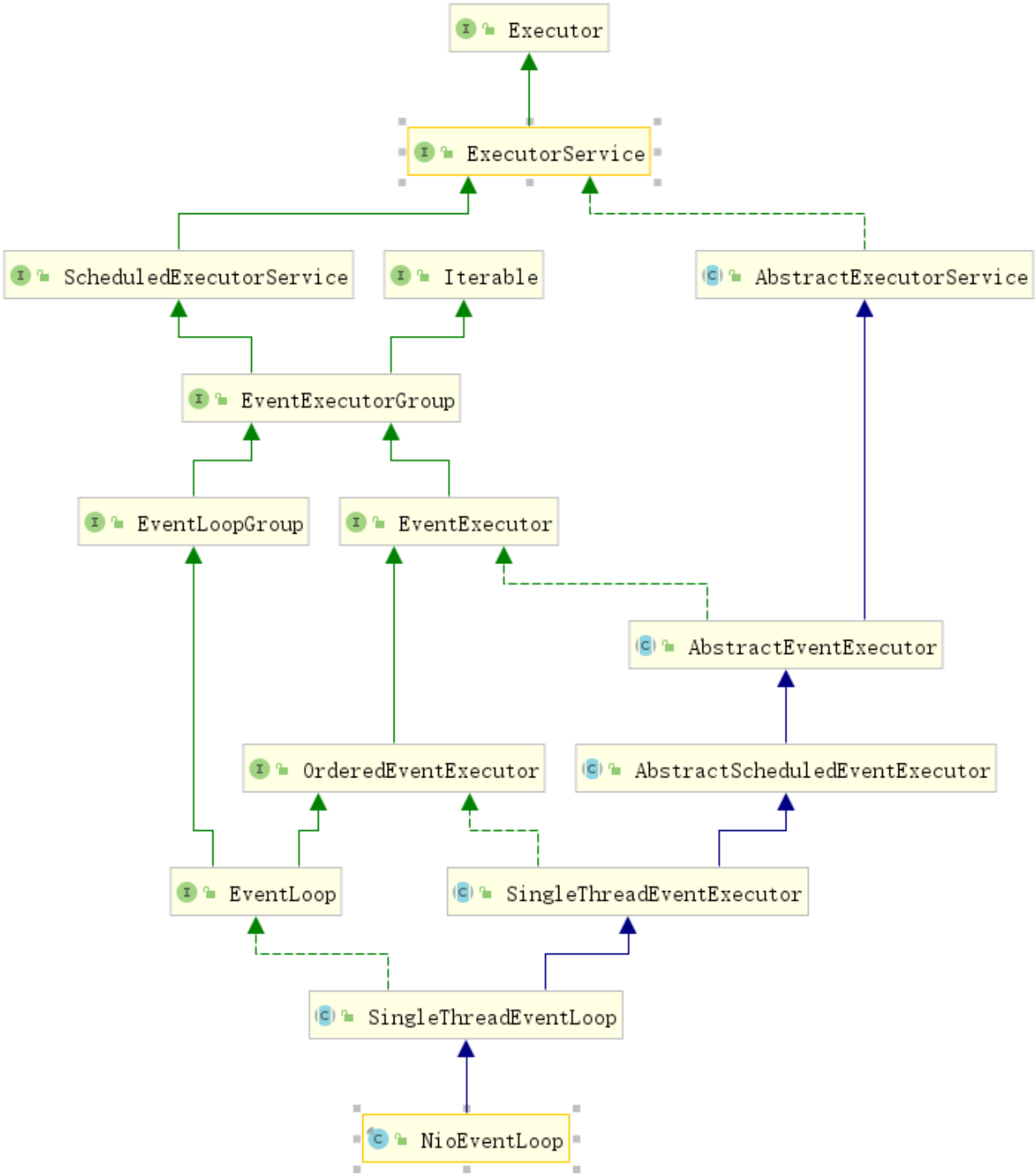
```
void init(Channel channel) {
        setChannelOptions(channel, newOptionsArray(), logger);
        setAttributes(channel, newAttributesArray());
        // 每个channel都有一个pipeline,这个地方是boss的pipeline
        ChannelPipeline p = channel.pipeline();
        // work线程组
        final EventLoopGroup currentChildGroup = childGroup;
        // work线程handler
        final ChannelHandler currentChildHandler = childHandler;

        final Entry<ChannelOption<?>, Object>[] currentChildOptions =
```

```
newOptionsArray(childOptions);
        final Entry<AttributeKey<?>, Object>[] currentChildAttrs =
newAttributesArray(childAttrs);

        p.addLast(new ChannelInitializer<Channel>() {
            @Override
            public void initChannel(final Channel ch) {
                final ChannelPipeline pipeline = ch.pipeline();
                // 此处得到的handler是外面配置的handler
                ChannelHandler handler = config.handler();
                if (handler != null) {
                    pipeline.addLast(handler);
                }
                // 这个地方就是放入一个处理accept事件的handler
                // ServerBootstrapAcceptor它肯定是一个inbound的处理器，因为此时是接收
                ch.eventLoop().execute(new Runnable() {
                    @Override
                    public void run() {
                        pipeline.addLast(new ServerBootstrapAcceptor(
                                ch, currentChildGroup, currentChildHandler,
currentChildOptions, currentChildAttrs));
                    }
                });
            }
        });
    }
```

看下NioEventLoop的关系图



- 具备几种能力
  - 定时任务
  - 异步线程
  - 注册多路复用

继续回到AbstractBootStrap类initAndRegister方法

```
// 这个里面会注册事件
ChannelFuture regFuture = config().group().register(channel);
if (regFuture.cause() != null) {
    if (channel.isRegistered()) {
        channel.close();
    } else {
        channel.unsafe().closeForcibly();
    }
}
```

SingleThreadEventLoop类中的register方法

```
  @Override
    public ChannelFuture register(final ChannelPromise promise) {
        ObjectUtil.checkNotNull(promise, "promise");
        //this就是NioEventLoop，可以就相当于一个线程
        promise.channel().unsafe().register(this, promise);
        return promise;
    }
```

AbstractChannel中的register中的register0方法

```
private void register0(ChannelPromise promise) {
    try {
        if (!promise.setUncancellable() || !ensureOpen(promise)) {
            return;
        }
        boolean firstRegistration = neverRegistered;
        // 这个方法就是注册事件
        doRegister();
        neverRegistered = false;
        registered = true;
        pipeline.invokeHandlerAddedIfNeeded();

        safeSetSuccess(promise);
        pipeline.fireChannelRegistered();
        if (isActive()) {
            if (firstRegistration) {
                pipeline.fireChannelActive();
            } else if (config().isAutoRead()) {
                // doRegister执行完成之后，会把原来注册的0改为accept事件
                beginRead();
            }
        }
    } catch (Throwable t) {
        closeForcibly();
        closeFuture.setClosed();
        safeSetFailure(promise, t);
    }
}
```

AbstractNioChannel类中的doRegister方法

```
protected void doRegister() throws Exception {
        boolean selected = false;
        for (;;) {
            try {
                //当向selector注册键值为0时 不会报错 但会什么都不做 所以 肯定后面在某个时刻会把
selector的键值改成正常的 那么怎么改呢? jdk 提供了这个api; SelectionKey#interestOps
                selectionKey = javaChannel().register(eventLoop().unwrappedSelector(), 0,
this);
                return;
            } catch (CancelledKeyException e) {
                if (!selected) {
                    eventLoop().selectNow();
                    selected = true;
                } else {
                    throw e;
                }
            }
        }
    }
```

回到dobind方法

```
 private ChannelFuture doBind(final SocketAddress localAddress) {
        final ChannelFuture regFuture = initAndRegister();
        final Channel channel = regFuture.channel();
        if (regFuture.cause() != null) {
            return regFuture;
        }

        if (regFuture.isDone()) {
            ChannelPromise promise = channel.newPromise();

            doBind0(regFuture, channel, localAddress, promise);
            return promise;
        } else {
            final PendingRegistrationPromise promise = new
PendingRegistrationPromise(channel);
            regFuture.addListener(new ChannelFutureListener() {
                @Override
                public void operationComplete(ChannelFuture future) throws Exception {
                    Throwable cause = future.cause();
                    if (cause != null) {
                        promise.setFailure(cause);
                    } else {
                        promise.registered();
                        // 调用doBind0方法
                        doBind0(regFuture, channel, localAddress, promise);
                    }
```

```
                }
            });
            return promise;
        }
    }
```

dobind0方法

```
  private static void doBind0(
            final ChannelFuture regFuture, final Channel channel,
            final SocketAddress localAddress, final ChannelPromise promise) {
        channel.eventLoop().execute(new Runnable() {
            @Override
            public void run() {
                if (regFuture.isSuccess()) {
                    //绑定端口
                    channel.bind(localAddress,
promise).addListener(ChannelFutureListener.CLOSE_ON_FAILURE);
                } else {
                    promise.setFailure(regFuture.cause());
                }
            }
        });
    }
```

ServerSocketChannel初始化过程

1.验证参数的合法性

2.通过channelFactory构建NioServerSocketChannel实例，而这个NioServerSocketChannel是根据ServerBootStrap初始化绑定的class关系

3.初始化NioServerSocketChannel中的handler,pipeline,这个pipeline会把服务端的handler添加到pipeleline中（init()）

4.register方法，将NioServerSocketChannel注册到NioEventLoop中的selector中，并且监听的事件是OP_ACCEPT

5.如果register注册成功，则调用ServerSocketChannel.bind方法，绑定ip和端口

NioEventLoopGroup

- 工作职责
  - 处理NioSocketChannel的IO就绪事件 select.select()
  - 异步任务，异步延迟任务
- NioEventLoop是一个线程，里面会有一个selector
  - 肯定会执行里面的run方法

源码的设计

- 接收到连接后要注册到某个NioEventLoop中
- 启动线程 用eventLoop.exectute()方法调用时，如果发现当前的eventLoop没有启动，则启动
- run方法中的实现逻辑
  - 调用select获取就绪的客户端连接
  - 处理异步任务
  - 处理异步延迟任务
- 任务执行
- new EventLoopGroup的时候只是创建了线程，并没有启动
- 再看channe.eventLoop().execute()

```
private static void doBind0(
    final ChannelFuture regFuture, final Channel channel,
    final SocketAddress localAddress, final ChannelPromise promise) {
    // 去执行任务，并且这个线程池里面的线程肯定是激活的状态，这个是线程池的原理
    channel.eventLoop().execute(new Runnable() {
        @Override
        public void run() {
            if (regFuture.isSuccess()) {
                channel.bind(localAddress,
promise).addListener(ChannelFutureListener.CLOSE_ON_FAILURE);
            } else {
                promise.setFailure(regFuture.cause());
            }
        }
    });
}
```

- SingleThreadEventExecutor这个类中的execute方法

```
public void execute(Runnable task) {
    ObjectUtil.checkNotNull(task, "task");
    execute(task, !(task instanceof LazyRunnable) && wakesUpForTask(task));
}
```

```
private void execute(Runnable task, boolean immediate) {
        boolean inEventLoop = inEventLoop();
        // 添加到任务列表
        addTask(task);
        if (!inEventLoop) {
            // 开启线程
            startThread();
            // 如果关闭了连接，则关闭连接
            if (isShutdown()) {
                boolean reject = false;
                try {
                    if (removeTask(task)) {
                        reject = true;
                    }
                } catch (UnsupportedOperationException e) {
```

```
                }
                if (reject) {
                    reject();
                }
            }
        }

        if (!addTaskWakesUp && immediate) {
            wakeup(inEventLoop);
        }
    }
```

- startThread方法

```
private void startThread() {
        if (state == ST_NOT_STARTED) {
            // 可能会存在并发的问题，用cas机制保证只有一个线程会去做这个事情
            if (STATE_UPDATER.compareAndSet(this, ST_NOT_STARTED, ST_STARTED)) {
                boolean success = false;
                try {
                    doStartThread();
                    success = true;
                } finally {
                    if (!success) {
                        STATE_UPDATER.compareAndSet(this, ST_STARTED,
ST_NOT_STARTED);
                    }
                }
            }
        }
    }
```

- doStartThread方法

```
private void doStartThread() {
        assert thread == null;
        executor.execute(new Runnable() {
            @Override
            public void run() {
                thread = Thread.currentThread();
                // 是否中断线程
                if (interrupted) {
                    thread.interrupt();
                }

                boolean success = false;
                updateLastExecutionTime();
                try {



                    // 因为eventLoop的父类SingleThreadEventExecutor是他，所以这个地方就是
```

```
EventLoop,所以当前SingleThreadEventExecutor类中肯定有一个未实现的run方法，等待在EventLoop中去实现
                    SingleThreadEventExecutor.this.run();
                    success = true;
                } catch (Throwable t) {
                    logger.warn("Unexpected exception from an event executor: ", t);
                }
        });
    }
```

- NioEventLoop中的run方法

```
 protected void run() {
        int selectCnt = 0;
        // 死循环
        for (;;) {
            try {
                int strategy;
                try {
                    // 这个地方是得到一个策略
                    // selectNow查询就绪的连接数量，立即返回，要么返回0.要么返回连接数量
                    strategy = selectStrategy.calculateStrategy(selectNowSupplier,
hasTasks());
                    switch (strategy) {
                    case SelectStrategy.CONTINUE:
                        continue;

                    case SelectStrategy.BUSY_WAIT:
                    // 代表任务队列为空
                    case SelectStrategy.SELECT:
                        long curDeadlineNanos = nextScheduledTaskDeadlineNanos();
                        // 代表没有定时任务，会调用select()一直阻塞
                        // 为什么有定时任务的时候，不能阻塞呢，因为比如我2分钟之后执行，而此时5分钟以内没有连接进来，那么一直阻塞的话，我的定时任务根本执行不了
                        if (curDeadlineNanos == -1L) {
                            curDeadlineNanos = NONE;
                        }
                        nextWakeupNanos.set(curDeadlineNanos);
                        try {
                            if (!hasTasks()) {
                                // 这个里面要么是当场唤醒，或者等待一段时间唤醒
                                // strategy:
                                //      小于等于0，说明没有连接，处理任务
                                //      大于0，说明有连接，处理连接
                                strategy = select(curDeadlineNanos);
                            }
                        } finally {

                            nextWakeupNanos.lazySet(AWAKE);
                        }
                    default:
                    }
                } catch (IOException e) {
```

```
                    rebuildSelector0();
                    selectCnt = 0;
                    handleLoopException(e);
                    continue;
                }

                selectCnt++;
                cancelledKeys = 0;
                needsToSelectAgain = false;
                // io使用时间的占比，默认50
                final int ioRatio = this.ioRatio;
                boolean ranTasks;
                // 为100，说明必须先去执行io任务
                if (ioRatio == 100) {
                    try {
                        if (strategy > 0) {
                            // 通过selector.select()->selectedKeys
                            // whiel(selectkeys){
                                    //如果有多个客户端连接进来
                                    // NioserverSocketChannel
                                        // -->pipeline(ServerBootstrapAcceptor)
                                        // -->handler
                            // }
                            processSelectedKeys();
                        }
                    } finally {
                        ranTasks = runAllTasks();
                    }
                } else if (strategy > 0) {
                    final long ioStartTime = System.nanoTime();
                    try {
                        processSelectedKeys();
                    } finally {
                        final long ioTime = System.nanoTime() - ioStartTime;
                        ranTasks = runAllTasks(ioTime * (100 - ioRatio) / ioRatio);
                    }
                } else {
                    // 说明当前队列中存在待执行的任务，并且没有就绪的channel
                    ranTasks = runAllTasks(0);
                }

                if (ranTasks || strategy > 0) {
                    if (selectCnt > MIN_PREMATURE_SELECTOR_RETURNS &&
logger.isDebugEnabled()) {
                        logger.debug("Selector.select() returned prematurely {} times
in a row for Selector {}.",
                                selectCnt - 1, selector);
                    }
                    selectCnt = 0;
                } else if (unexpectedSelectorWakeup(selectCnt)) {
                    selectCnt = 0;

                }
```

```
            } catch (CancelledKeyException e) {
                if (logger.isDebugEnabled()) {
                    logger.debug(CancelledKeyException.class.getSimpleName() + "
raised by a Selector {} - JDK bug?",
                            selector, e);
                }
            } catch (Error e) {
                throw (Error) e;
            } catch (Throwable t) {
                handleLoopException(t);
            } finally {
                try {
                    if (isShuttingDown()) {
                        closeAll();
                        if (confirmShutdown()) {
                            return;
                        }
                    }
                } catch (Error e) {
                    throw (Error) e;
                } catch (Throwable t) {
                    handleLoopException(t);
                }
            }
        }
    }
```

processSelectedKeys方法分析

```
private void processSelectedKeys() {
    if (selectedKeys != null) {
        // 优化
        processSelectedKeysOptimized();
    } else {
        // 没有做优化
        processSelectedKeysPlain(selector.selectedKeys());
    }
}
```

没有优化processSelectedKey方法

```
private void processSelectedKey(SelectionKey k, AbstractNioChannel ch) {
        final AbstractNioChannel.NioUnsafe unsafe = ch.unsafe();
        if (!k.isValid()) {
            final EventLoop eventLoop;
            try {
                eventLoop = ch.eventLoop();
            } catch (Throwable ignored) {

                return;

            }
```

```
            if (eventLoop == this) {
                unsafe.close(unsafe.voidPromise());
            }
            return;
        }

        try {
            int readyOps = k.readyOps();
            // 如果是连接，处理连接事件
            if ((readyOps & SelectionKey.OP_CONNECT) != 0) {
                int ops = k.interestOps();
                ops &= ~SelectionKey.OP_CONNECT;
                k.interestOps(ops);

                unsafe.finishConnect();
            }
            // 如果是写，处理写事件
            if ((readyOps & SelectionKey.OP_WRITE) != 0) {
                ch.unsafe().forceFlush();
            }
            // 如果是读，处理读事件
            if ((readyOps & (SelectionKey.OP_READ | SelectionKey.OP_ACCEPT)) != 0 ||
readyOps == 0) {
                unsafe.read();
            }
        } catch (CancelledKeyException ignored) {
            unsafe.close(unsafe.voidPromise());
        }
    }
```

processSelectedKeysOptimized优化方法

```
// 把hashSet变成了数组，优化了hash冲突和时间复杂度为O(1)的add方法
private void processSelectedKeysOptimized() {
    for (int i = 0; i < selectedKeys.size; ++i) {
        // 这个地方拿到的selectKeys是一个数组
        final SelectionKey k = selectedKeys.keys[i];
        selectedKeys.keys[i] = null;
        final Object a = k.attachment();

        if (a instanceof AbstractNioChannel) {
            processSelectedKey(k, (AbstractNioChannel) a);
        } else {
            @SuppressWarnings("unchecked")
            NioTask<SelectableChannel> task = (NioTask<SelectableChannel>) a;
            processSelectedKey(k, task);
        }

        if (needsToSelectAgain) {
            selectedKeys.reset(i + 1);
            selectAgain();

            i = -1;
```

```
        }
    }
}
```

SelectedSelectionKeySet类

```
SelectionKey[] keys;
// 这个地方的调用是通过反射，是无法找到的，回去看SelectedSelectionKeySet是如何初始化的
// 经过替换后，只要发生IO操作，就会把调用add方法把SelectionKey添加到keys中
public boolean add(SelectionKey o) {
    if (o == null) {
        return false;
    }

    keys[size++] = o;
    if (size == keys.length) {
        increaseCapacity();
    }

    return true;
}
```

在NioEventLoop类中的openSelector方法

```
private SelectorTuple openSelector() {
        final Selector unwrappedSelector;
        try {
            unwrappedSelector = provider.openSelector();
        } catch (IOException e) {
            throw new ChannelException("failed to open a new selector", e);
        }

        if (DISABLE_KEY_SET_OPTIMIZATION) {
            return new SelectorTuple(unwrappedSelector);
        }
        // 得到一个SelectorImpl对象，这个SelectorImpl里面有一个HashSet<SelectionKey> keys
        Object maybeSelectorImplClass = AccessController.doPrivileged(new
PrivilegedAction<Object>() {
            @Override
            public Object run() {
                try {
                    return Class.forName(
                            "sun.nio.ch.SelectorImpl",
                            false,
                            PlatformDependent.getSystemClassLoader());
                } catch (Throwable cause) {
                    return cause;
                }
            }
        });
```

```
        if (!(maybeSelectorImplClass instanceof Class) ||
            !((Class<?>)
maybeSelectorImplClass).isAssignableFrom(unwrappedSelector.getClass())) {
            if (maybeSelectorImplClass instanceof Throwable) {
                Throwable t = (Throwable) maybeSelectorImplClass;
                logger.trace("failed to instrument a special java.util.Set into: {}",
unwrappedSelector, t);
            }
            return new SelectorTuple(unwrappedSelector);
        }

        final Class<?> selectorImplClass = (Class<?>) maybeSelectorImplClass;
        final SelectedSelectionKeySet selectedKeySet = new SelectedSelectionKeySet();

        Object maybeException = AccessController.doPrivileged(new PrivilegedAction<Object>
() {
            @Override
            public Object run() {
                try {
                    // 通过反射得到SelectorImpl里面的Set<SelectionKey> selectedKeys字段
                    Field selectedKeysField =
selectorImplClass.getDeclaredField("selectedKeys");
                     // 通过反射得到SelectorImpl里面的Set<SelectionKey> publicSelectedKeys
                    Field publicSelectedKeysField =
selectorImplClass.getDeclaredField("publicSelectedKeys");

                    if (PlatformDependent.javaVersion() >= 9 &&
PlatformDependent.hasUnsafe()) {
                        // 得到selectedKeysField属性和publicSelectedKeysField属性的偏移量
                        long selectedKeysFieldOffset =
PlatformDependent.objectFieldOffset(selectedKeysField);
                        long publicSelectedKeysFieldOffset =

 PlatformDependent.objectFieldOffset(publicSelectedKeysField);
                        // 相当于把SelectorImpl类中的两个属性改了
                        //Set<SelectionKey> selectedKeys ->改成了SelectedSelectionKeySet,
里面的selectkeys是一个数组，原来是一个hashSet
                        //Set<SelectionKey> publicSelectedKeys ->改成了
SelectedSelectionKeySet
                        if (selectedKeysFieldOffset != -1 && publicSelectedKeysFieldOffset
!= -1) {
                            PlatformDependent.putObject(
                                    unwrappedSelector, selectedKeysFieldOffset,
selectedKeySet);
                            PlatformDependent.putObject(
                                    unwrappedSelector, publicSelectedKeysFieldOffset,
selectedKeySet);
                            return null;
                        }
                    }



                    Throwable cause = ReflectionUtil.trySetAccessible(selectedKeysField,
```

```
true);
                    if (cause != null) {
                        return cause;
                    }
                    cause = ReflectionUtil.trySetAccessible(publicSelectedKeysField,
true);
                    if (cause != null) {
                        return cause;
                    }

                    selectedKeysField.set(unwrappedSelector, selectedKeySet);
                    publicSelectedKeysField.set(unwrappedSelector, selectedKeySet);
                    return null;
                } catch (NoSuchFieldException e) {
                    return e;
                } catch (IllegalAccessException e) {
                    return e;
                }
            }
        });

        if (maybeException instanceof Exception) {
            selectedKeys = null;
            Exception e = (Exception) maybeException;
            logger.trace("failed to instrument a special java.util.Set into: {}",
unwrappedSelector, e);
            return new SelectorTuple(unwrappedSelector);
        }
        selectedKeys = selectedKeySet;
        logger.trace("instrumented a special java.util.Set into: {}", unwrappedSelector);
        return new SelectorTuple(unwrappedSelector,
                                 new SelectedSelectionKeySetSelector(unwrappedSelector,
selectedKeySet));
    }
```

select/poll/epoll模型的区别

1、支持一个进程所能打开的最大连接数

select：单个进程所能打开的最大连接数有FD_SETSIZE宏定义，其大小是32个整数的大小（在32位的机器上，大小就是3232，同理64位机器上FD_SETSIZE为3264），当然我们可以对进行修改，然后重新编译内核，但是性能可能会受到影响，这需要进一步的测试。

poll：poll本质上和select没有区别，但是它没有最大连接数的限制，原因是它是基于链表来存储的。

epoll：虽然连接数有上限，但是很大，1G内存的机器上可以打开10万左右的连接，2G内存的机器可以打开20万左右的连接。

2、FD剧增后带来的IO效率问题

select：因为每次调用时都会对连接进行线性遍历，所以随着FD的增加会造成遍历速度慢的“线性下降性能问题”。

poll：同上

epoll：因为epoll内核中实现是根据每个fd上的callback函数来实现的，只有活跃的socket才会主动调用callback，所以在活跃socket较少的情况下，使用epoll没有前面两者的线性下降的性能问题，但是所有socket都很活跃的情况下，可能会有性能问题。

3、 消息传递方式

select：内核需要将消息传递到用户空间，都需要内核拷贝动作

poll：同上

epoll：epoll通过内核和用户空间共享一块内存来实现的。

综上，在选择select，poll，epoll时要根据具体的使用场合以及这三种方式的自身特点。

1、表面上看epoll的性能最好，但是在连接数少并且连接都十分活跃的情况下，select和poll的性能可能比epoll好，毕竟epoll的通知机制需要很多函数回调。

select，poll实现需要自己不断轮询所有fd集合，直到设备就绪，期间可能要睡眠和唤醒多次交替。而epoll其实也需要调用epoll_wait不断轮询就绪链表，期间也可能多次睡眠和唤醒交替，但是它是设备就绪时，调用回调函数，把就绪fd放入就绪链表中，并唤醒在epoll_wait中进入睡眠的进程。虽然都要睡眠和交替，但是select和poll在“醒着”的时候要遍历整个fd集合，而epoll在“醒着”的时候只要判断一下就绪链表是否为空就行了，这节省了大量的CPU时间。这就是回调机制带来的性能提升。

2、select低效是因为每次它都需要轮询。但低效也是相对的，视情况而定，也可通过良好的设计改善

select，poll每次调用都要把fd集合从用户态往内核态拷贝一次，并且要把current往设备等待队列中挂一次，而epoll只要一次拷贝，而且把current往等待队列上挂也只挂一次（在epoll_wait的开始，注意这里的等待队列并不是设备等待队列，只是一个epoll内部定义的等待队列）。这也能节省不少的开销。

单机百万连接性能调优

通常启动一个服务端会绑定一个端口，例如8080，客户端连接时都会分配一个端口，在操作系统中最大的端口号是65535，那么1024及以下的端口号是给操作系统本身用的，所以实际的就剩下1025-65535这么多端口，还有一些常用的端口，那么可用的端口接近6万左右，那么服务端开启一个端口，可以支撑客户端6万的连接（理论上）

如果在服务端开启100个端口，比如[8000,8100）100个端口，那么理论上可以支撑6万*100=600万的连接

我们知道在linux系统下，使用ulimit -n这样的命令可以查看一个进程在linux中一个进程能够打开的最大文件数，开启一个TCP连接就会在linux系统中创建一个文件，所以就是受到最大文件数限制

- 突破这个文件限制，打开/etc/security/limits.conf文件

  ```
  sudo vi /etc/security/limits.conf
  ```

  在这个文件末尾加两行代码

  ```
  * hard nofile 1000000
  * soft nofile 1000000
  ```

  突破全局文件句柄

```
cat /proc/sys/fs/file-max
```

然后再修改全局的文件句柄限制

```
sudo vi /etc/sysctl.conf
```

修改fs.file-max=1000000

```
fs.file-max=1000000
```

还有一种是业务代码调优，如果业务代码执行太慢，也会导致性能，所以业务代码可以再弄一个线程池可以执行业务代码

netty面试题

- Netty 是什么？
  - Netty 是一个 基于 NIO 的 client-server(客户端服务器)框架，使用它可以快速简单地开发网络应用程序。
  - 它极大地简化并优化了 TCP 和 UDP 套接字服务器等网络编程,并且性能以及安全性等很多方面甚至都要更好。
  - 支持多种协议 如 FTP，SMTP，HTTP 以及各种二进制和基于文本的传统协议。
  - 很多开源项目比如我们常用的 Dubbo、RocketMQ、Elasticsearch、gRPC 等等都用到了 Netty。
- 为什么要用 Netty？
  - 因为 Netty 具有下面这些优点，并且相比于直接使用 JDK 自带的 NIO 相关的 API 来说更加易用。
  - 统一的 API，支持多种传输类型，阻塞和非阻塞的。
  - 简单而强大的线程模型。
  - 自带编解码器解决 TCP 粘包/拆包问题。
  - 自带各种协议栈。
  - 真正的无连接数据包套接字支持。
  - 比直接使用 Java 核心 API 有更高的吞吐量、更低的延迟、更低的资源消耗和更少的内存复制。
- Netty 应用场景了解么？
  - 作为 RPC 框架的网络通信工具
  - 实现一个自己的 HTTP 服务器
  - 实现一个即时通讯系统
  - 实现消息推送系统
- Netty 核心组件有哪些？分别有什么作用？
  - Channel

    Channel 接口是 Netty 对网络操作抽象类，它除了包括基本的 I/O 操作，如 bind()、connect()、read()、write() 等。

    比较常用的Channel接口实现类是NioServerSocketChannel（服务端）和NioSocketChannel（客户端），这两个 Channel 可以和 BIO 编程模型中的ServerSocket以及Socket两个概念对应上。Netty 的 Channel 接口所提供的 API，大大地降低了直接使用 Socket 类的复杂性。
  - EventLoop
  - ChannelFuture

    - ChannelHandler 和 ChannelPipeline
    - ByteBuf
- EventloopGroup 了解么?和 EventLoop 啥关系?

  线程组和线程的关系
- NioEventLoopGroup 默认的构造函数会起多少线程?

  cpu核心数*2
- Netty 线程模型了解么?
    - 单线程
    - Reactor多线程模型
    - Reactor主从多线程模型
- 如何处理粘包黏包（4中方式都可以）
    - 固定长度，例如120字节代表一个整体的包信息，不足的就补位，解码器在处理这类定长消息的时候比较简单，每次读到指定长度的字节后再进行解码。
    - 通过回车换行符区分消息，例如http,这类区分消息的方式多用于文本协议
    - 通过特定的分隔符区分整包的消息
    - 通过在协议头/消息头中设置长度属性来标识整包消息
- Netty 长连接、心跳机制了解么?
    - 在 TCP 保持长连接的过程中，可能会出现断网等网络异常出现，异常发生的时候， client 与 server 之间如果没有交互的话，它们是无法发现对方已经掉线的。为了解决这个问题, 我们就需要引入 **心跳机制。**

      心跳机制的工作原理是: 在 client 与 server 之间在一定时间内没有数据交互时, 即处于 idle 状态时, 客户端或服务器就会发送一个特殊的数据包给对方, 当接收方收到这个数据报文后, 也立即发送一个特殊的数据报文, 回应发送方, 此即一个 PING-PONG 交互。所以, 当某一端收到心跳消息后, 就知道了对方仍然在线, 这就确保 TCP 连接的有效性.

      TCP 实际上自带的就有长连接选项，本身是也有心跳包机制，也就是 TCP 的选项：SO_KEEPALIVE。但是，TCP 协议层面的长连接灵活性不够。所以，一般情况下我们都是在应用层协议上实现自定义心跳机制的，也就是在 Netty 层面通过编码实现。通过 Netty 实现心跳机制的话，核心类是 IdleStateHandler
- Netty 的零拷贝了解么?
    - Netty 中的零拷贝体现在以下几个方面：

      1.使用 Netty 提供的 CompositeByteBuf 类, 可以将多个ByteBuf 合并为一个逻辑上的 ByteBuf, 避免了各个 ByteBuf 之间的拷贝。

      2.ByteBuf 支持 slice 操作, 因此可以将 ByteBuf 分解为多个共享同一个存储区域的 ByteBuf, 避免了内存的拷贝。

      3.通过 FileRegion 包装的FileChannel.tranferTo 实现文件传输, 可以直接将文件缓冲区的数据发送到目标 Channel, 避免了传统通过循环 write 方式导致的内存拷贝问题.
- netty中boss线程池设置为多个，会有多个boss线程吗
    - Netty boss 线程池是处理 accept事件的，不管线程池多大，只会使用一个线程，
- 为什么只使用一个线程，还要用到线程池呢?
    - 主要是异常的情况下，线程die了，可以再创建一个新线程，
- 那什么情况下boss线程池可以使用多个线程呢?

    - 那就是当ServerBootstrap bind多个端口时。每个端口都有一个线程eventloop accept事件。
- Netty的高性能及NIO的epoll空轮询bug（Java nio 空轮询bug也就是Java nio在Linux系统下的epoll空轮询问题。）
    - select.select()方法可能会有返回值，这时操作系统层面的问题
    - 关注的select轮询事件返回数量为0，NIO照样不断的从select本应该阻塞的`Selector.select()/Selector.select(timeout)`中wake up出来，导致CPU 100%问题
    - 若Selector的轮询结果为空，也没有wakeup或新消息处理，则发生空轮询，CPU使用率100%
    - 解决方法：

        >
        > - 1.对Selector的select操作周期进行统计，每完成一次空的select操作进行一次计数，
        > - 2.若在某个周期内连续发生N次空轮询，则触发了epoll死循环bug。
        > - 3.重建Selector，判断是否是其他线程发起的重建请求，若不是则将原SocketChannel从旧的Selector上去除注册，重新注册到新的Selector上，并将原来的Selector关闭。
- 无锁化的串行设计理念
    - 通过串行化设计，即消息的处理尽可能在同一个线程内完成，期间不进行线程切换，这样就避免了多线程竞争和同步锁。
    - Netty采用了串行无锁化设计，在IO线程内部进行串行操作，避免多线程竞争导致的性能下降。表面上看，串行化设计似乎CPU利用率不高，并发程度不够。但是，通过调整NIO线程池的线程参数，可以同时启动多个串行化的线程并行运行，这种局部无锁化的串行线程设计相比一个队列-多个工作线程模型性能更优。